

Technical Documentation

RAG-Based Customer Support Assistant

Using LangGraph & HITL

Submitted by: Roshini Parvathi

Batch: Feb 2026

Organization: Innomatics Research Labs

Date: 23/04/2026

TABLE OF CONTENTS

1. Introduction

2. Architecture

3. Design Decisions

4. Workflow

5. Logic

6. HITL

7. Challenges

8. Testing

9. Future Scope



TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant (LangGraph + Groq)

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines:

- **Information Retrieval** (fetch relevant data)
- **Text Generation** (LLM generates answer)

Instead of relying only on model memory, RAG uses external data sources (like PDFs).

Why RAG is Needed

- Reduces hallucinations
 - Provides factual answers
 - Enables domain-specific responses
 - Improves explainability via citations
-

Use Case

Customer Support Assistant that:

- Answers queries from a knowledge base PDF
 - Retrieves relevant sections
 - Generates contextual responses
 - Escalates to human if needed
-

2. System Architecture Explanation

The system consists of:

1. Input Layer

- User enters query via Streamlit UI

2. Document Processing

- PDF is loaded
- Content split into sections

3. Embedding Layer

- Text converted into vectors using HuggingFace embeddings

4. Vector Database

- ChromaDB stores embeddings
- Enables fast similarity search

5. Retrieval Layer

- Fetches top-k relevant chunks

6. LLM Layer

- Groq API (LLaMA3 model) generates answers

7. Workflow Layer

- LangGraph controls execution:
 - Process → Route → Output

8. HITL Layer

- Escalates to human if answer not found
-

3. Design Decisions

Chunking Strategy

- Section-based splitting using numbered headings
 - Ensures semantic alignment
-

Embedding Strategy

- HuggingFace embeddings
 - Lightweight and effective
-

Retrieval Approach

- Top-k similarity search
 - Ensures relevant context
-

Prompt Design

- Strict instructions:
 - Use only context
 - Avoid hallucination
 - Handle multi-part queries
-

4. Workflow Explanation (LangGraph)

Node 1: Process

- Retrieves documents
 - Generates answer using LLM
-

Node 2: Route

- Checks:
 - Context availability
 - LLM response
-

Node 3: HITL

- Returns escalation response
-

State Transition

Query → Process → Route → (Answer / HITL)

5. Conditional Logic

Intent Detection

- Based on query keywords and response
-

Routing Decision

- If context exists → Answer
 - If no context → HITL
 - If LLM says “I don’t know” → HITL
-

6. HITL Implementation

Role of Human

- Handles unresolved queries
 - Ensures reliability
-

Trigger Conditions

- No relevant data
 - Complex query
 - User explicitly requests human
-

Benefits

- Improves trust
 - Reduces failure cases
-

Limitations

- Requires manual intervention
- Not scalable without automation

7. Challenges & Trade-offs

1. Retrieval Accuracy vs Speed

- More chunks → better accuracy
 - But increases latency
-

2. Chunk Size vs Context Quality

- Small chunks → precise
 - Large chunks → better context
-

3. Cost vs Performance

- Faster models cost more
 - Trade-off between speed and cost
-

8. Testing Strategy

Test Types

Functional Tests

- Refund policy
 - Account issues
-

Semantic Tests

- Paraphrased queries
-

Multi-intent Tests

- Reset password + contact support
-

Edge Cases

- Unknown queries

HITL Tests

- “Talk to human”
-

9. Future Enhancements

- Multi-document support
- Feedback loop (learning system)
- Memory-based conversations
- API deployment
- UI improvements